

Identifiers for Pattern Matching

Document #: P2941R0
Date: 2022-02-24
Project: Programming Language C++
SG17
Reply-to: Mihail Naydenov
<mihailnaydenov@gmail.com>

§ 1 Abstract

This paper investigates the challenges we face with identifiers inside Pattern Matching (PM). It also proposes a method for their introduction, one that is inline with current practices, without new language syntax.

§ 2 Challenges

One of the many challenges in PM has to do with identifiers. There is an unavoidable ambiguity in what we mean by a variable named `x`: Is it a newly introduced entity or a reference to an existing one? There is no right answer and each system decides on its own. For example in Python, `x` is a new name and if the user wants to access an existing one it must contain a dot (essentially to be a path) - `something.x`. In Rust also `x` is a new name. Swift and C# introduce new names the same way this is done outside PM - with `let` and `var` respectively.

Variable introduction style might seem like a minor, “esthetics” issue, but is really not, especially in C++, for two important reasons:

- First, in C++ *there is no variable introduction keyword*. This is in contrast to the above mentioned Swift and C#.
- Second, *types and variables can have the same name* - `class s{}; s s; .`

These two just add up to the preexisting ambiguities inside PM, creating an even more complicated context.

§ 2.0.1 So many identifiers, so little syntax

Let’s examine the number of possibilities for what an identifier could mean in the C++ pattern matching scenario. We mentioned some of them already:

- `x` can be a PM variable, a binding
- `x` can be a non-PM variable, one to compare against

- `x` can be a type

Before continuing, here are our current solutions for the above 3.

The main PM proposal, p1371¹, as of its latest iteration at the time of writing:

- creating a binding is the default, with the explicit limitation, one can not bind an arbitrary pattern. This is, only the last value, after all matching is done, can be captured behind the name `x`.
- non-PM variables must be proceeded with `case - case x`.
- types are used inside the “Alternative Pattern”, which surrounds their names with angle brackets - `<x>`.

However, according to p2688², the above will be revisited in future versions and will be as follows:

- creating a binding will be done via a `let` keyword.
- non-PM variables will be “just expression” and used directly, making the “non-PM variable for comparison” the default.
- (“Alternative Pattern” is unchanged.)

The “is as” proposal (p2392)³ is as follows:

- binding is created if the identifier is to the left of `is` or `as`.
- non-PM variables are the ones that appear to the right of `is`.
- types *also* appear to the right, same as non-PM variables.

The problem with type selector

It is worth taking a special attention to type selector in our current approaches.

Both current solutions accept an expression *and* a typename behind the same syntax:

Main Proposal

```
constexpr auto size = 13;           using size = int;

inspect (some_value) {               inspect (some_value) {
  <size> => // select a some_value    <size> => // select a some_value
        alternative, using the      alternative, using the type
        value 13                    int
};                                   };

```

Exact same syntax in both cases

Is As

<code>constexpr auto size = 13;</code>	<code>using size = int;</code>
<code>inspect (some_value) {</code>	<code>inspect (some_value) {</code>
<code> is size => // some_value is equal</code>	<code> is size => // some_value is of type</code>
<code> to the value 13</code>	<code> int</code>
<code>};</code>	<code>};</code>

Exact same syntax in both cases

This situation is not perfect because it makes the PM expression *fragile* - one extra variable definition (or `using enum`), anywhere prior to the PM, *could change its meaning*.

```
struct size{...};
enum class Selector { size, ...};
...
void function() {
// constexpr auto size = 13; //< if someone, or an include, adds this, in
    either global or local scope,
    // the definition of the PM will change. It
    might even still compile, but give a different answer.

// using enum Selector;    //< same here

    ...

    inspect (some_value) {
        <size> => // We can't really be sure, a selection based on type is
                made
    };
}
```

In a bigger function, it is not hard to imagine this scenario arising.

Also note, it might happen in reverse. Both a `size` type and a `size` variable could already be defined, after all there is no issue doing so, and someone might write an PM expression, being aware only of one of the definitions. He/she will scratch their head why the PM is not doing what it is expected, or worse, appear to do so but really not.

Why this is not an issue today?

In general, there are effectively no places where there is a competition b/w type and variable, a place where one must explicitly select one or another in order to *choose and have a different result*. We must specifically *create* such scenarios, like a function which takes either a non type or type. However, and this is the big difference to PM, such a function will be an API that we code against. That API will either be such from the beginning, and we take precaution when using it, or it will evolve into such, then it will/should be documented of the hazard when upgrading.

This is not the case with PM, and it should not be. PM is a *native expression*, not an API to some library. *Expressions in C++ do not switch meaning, based on the presence (or not) of type and variable with the same name*. They might fail to compile, but will not change

meaning, *there are no alternative implementations*. PM should behave no worse than that, hopefully better. If not, we create a “foot gun”, for which we must write yet another guide on how to avoid (“always use different naming conventions for type and variable”, “always use elaborate type specifier in PM”, etc). We don’t want that.

As you can see, things are already not easy with these 3 cases, but there is more:

- `x` can be an existing, non-PM variable, used for binding, an assignment.

Arguably, this last case mainly concerns patterns *outside* PM. For example, we can imagine extending structured bindings to bind to existing variables:

```
some_t a,b;  
[a,b] = something; //< bind to preexisting variables
```

Although, there is `std::tie`, it’s a hidden, non-obvious way of doing things. More importantly, it is *not composable* with any patterns:

```
some_t a,b;  
// some-pattern below is a hypothetical pattern-used-outside-PM.  
some-pattern std::tie(some-pattern a, b) = something; //< can't bind to  
preexisting variables from within patterns
```

Outside PM we will certainly need to bind to existing variables, but binding to existing variables *inside* PM could also be beneficial. We might want to fill a preexisting data structure directly. Something along the lines of:

```
void useSize(size_t& size);  
  
size_t useAndReturnSize() {  
    size_t size{};  
    inspect(some_rect) {  
        [__, let sz = {5,5}] => useSize(size = sz);  
        ...  
    }  
    ...  
    return size;  
};
```

Here, *ideally* we would have just used the `size` variable directly. `sz` is just a crutch to carry us around the fact, we can’t bind directly to preexisting variables. Granted, this use is not that important exactly because the workaround is easy and as such, current proposals do not address this usage.

There is however, much more important case, missing from the discussions so far.

We talk about introducing a variable for PM, a binding, but this binding should have control of its mutability, much like any other variable in the language. We should have both a binding for reading and a binding for writing.

That scenario is not handled directly by the current proposals and a simplified approach is taken, where *all bindings are mutable*, unless the object we “inspect” is `const`. In that case

all binding are const by the regular constness preservation rules. Needless to say, this is far from perfect for two reasons:

- Pattern Matching, in contrast to `switch` and `if-else` is recursive, which results in a lot of complexity, with many branches in an overall three structure. We will absolutely encounter the need to have *almost* all branches be const, but *some* be mutable simply because not all paths will have the same requirements and usage. An “all or nothing” approach will be suboptimal, hurting const correctness overall - if you want that one variable, in that one arm to be mutable, *all* of them will have to be, the root can no longer be const.
- Making the root const will have no effect on the mutability of the data if the data is accessed indirectly (pointer, view). Considering the widespread use of such objects in C++, both modern and classical, this will be a half-measure or even useless. A practical mutability can only be achieved by having it be defined as part of the particular binding and not for the containing object we inspect.

At this point it should be evident, there are plenty of complications regarding identifiers in the PM context, with many potential uses and very limited ways to express them. Here is the complete list:

- `x` can be a PM variable, used for observation
- `x` can be a PM variable, used for modification
- `x` can be a non-PM variable, one to compare against
- `x` can be a non-PM variable, one to write to
- `x` can be a type

Of course, not all uses are equally important, but all must be taken into account as part of the overall requirements and a view into “the bigger picture”. Otherwise we *will* write ourselves into a corner, as the syntax options are extremely limited.

§ 2.1 Type selector

Identifiers for type selector need a special mention. If we step back and consider what syntax we would *like* to see as a natural way of doing type switching, we should look no further then the very first proposals for Pattern Matching like pattern-matching-November-2014⁴ or the early p0095.⁵ They both advertise something like:

```
Some{...}
```

to select (and deconstruct) a type `Some` from a polymorphic source value. This makes sense - deconstructing is the opposite of constructing and often programming languages use a syntax, very similar to either class definition or construction:

Rust class definition

```
... Some{...}
```

Rust class deconstruction

```
... Some{...}
```

C# class definition

```
... Some{...}
```

C# class deconstruction

```
... Some{...}
```

Python class construction

```
... Some(...)
```

Python class deconstruction

```
... Some(...)
```

Scala class definition

```
... Some(...)
```

Scala class deconstruction

```
... Some(...)
```

In other words, our initial intention to have deconstruction mirroring construction was more or less “right”. Something however happen in the process and we drifted away of it. Using square instead of curly brackets for deconstruction is understandable, considering Structured Bindings are already in the language. However, we also lost the natural way of expressing the desired type:

Main Proposal

```
<Some>[...]
```

extra angle brackets

Is As Proposal

```
[...] as Some
```

a new `as` idiom

The `as`, although looking nice, is foreign to C++. To change this fact, we must alter the language, adding `as` outside PM also. On the other hand, angle brackets can hardly be perceived as pretty or natural and, considering the recursive nature of PM, will bring considerable visual noise.

§ 3 The Bigger Picture

This is an overview, including future directions. For the concretely proposed items see next section.

In the previous section we made the observation, an identifier can be one of those:

- a binding, creating a new variable
- a binding, reusing an existing variable
- an existing variable to compare against
- an existing variable to write to

— a type name

It is clear, it is not possible to have a simple, single identifier without encountering multiple ambiguities. To account for that, we must “mark” identifiers for their specific use. In the main proposal this is done by either using `case` for “existing variable to compare against” (p1371) or with `let` (p2688) to mark new identifiers. Angle brackets are used around type names. In the “is as” proposal, the location of the identifier in relation to `is` or `as` is the determining factor.

Regarding the latter, we can make the observation, `is` is quite similar to an operator, we already have, the `operator==`. This means, marking values for comparison with an `==` can be a reasonable approach as well, one that does not need introducing new language features:

```
int value = 5;
int number = ...;

inspect (number) {
  ==value => //< matches `value`
  // or !=value if we want!
  ...
};
```

This looks promising, the code is self-explanatory, free from ambiguity, though arguably a bit verbose. Later, in the proposal part, this issue is addressed as well.

Please note, `inspect`, and much of the syntax, is just a placeholder, reusing it from the main PM proposal (p1371). The authors seem to be moving away from it in p2688, but it can still serve us here.

Also, having `==` does *not* exclude intruding `is` in the future. `is` will be just another operator, both in and outside PM.

One very interesting side effect is that by using `==` (or possibly `!=`), we can also *naturally* introduce the other comparison operators:

```
inspect (number) {
  >=value => //< greater or equal
  <=value => //< less or equal
  < value => //< less then
  > value => //< greater then
  ...
};
```

This is a really nice bonus feature, already available in C# PM for example.

Moving forward. We saw how we can introduce existing identifiers for comparison naturally, now let's get to the much harder cases - introducing new variables. In C++, much like C before that, *there is no special keyword or any other syntactical marking*. Identifiers for variables are introduced by a preceding identifier of a type:

```
id_of_type id_of_var;
```

`id_of_var` is new variable, indicated only by the fact, it has a preceding type `id_of_type`.

This is a though place to be as this kind of introduction is completely unusable for PM. The route current proposals take is to introduce a new language syntax. This is either special mini language, in the main PM proposal case, where having new variables are introduced with the `let` keyword. Or, in the “is as” proposal, by having new language feature, the `is` and `as` operators, which in PM can create new variables among other things. Having to introduce a new language is high price to pay and it leads to inconsistencies - in and outside PM the variables are introduced differently. Faced with the issue, we must either address it by changing the rest of the language (the “is as” proposal) or ignored it (main proposal).

The described is not the case with languages, which *already* have a keyword for variable introduction. Swift using `let` inside PM is *fundamentally* different then C++ doing it - `let` is already the way to introduce variables. Same for C# and its `var` keyword. C++ using `let` *just for PM* creates an “one-off” within the language in a place where semantically there is no new functionality - bindings are still variables, just in a different context.

This negativistic scenario is not the only one possible however.

Interestingly, having to precede the identifier with a type has one notable exception - the lambda capture. There we don’t have to supply the type before a new variable:

```
[a=something] { // New variable `a` is introduced for the lambda with the
                same value as `something`.
    ...
}
```

This is used to make the syntax *concise* and it works because there are *no uninitialized captures*.

Coincidentally, *both* of those are true for the PM as well - we need a concise syntax *and* there are no uninitialized bindings!

Reusing the syntax seems like a very sensible option:

```
int number = ...;

inspect (number) {
    n=__ => // new binding `n` is introduced for the PM with same value as
           the matched ::number
    // or just n= omitting the __
    ...
};
```


Similarly how we create a *lambda-local* variable, *always* initialized to some preexisting value, we can create a *PM-local* variable, again *always* initialized to a preexisting value. This time however the value is coming *implicitly* from the object we “inspect” instead of assigning it manually.

In the above example, we might continue matching, adding an expected value:

```
int number = ...;

inspect (number) {
  n=5 => // new binding `n` is introduced for the PM, iff ::number is 5
  ...
};
```

Here we create a binding `n`, equal to `5`. Of course, it is not we that “assign” `5` to the binding, in contrast to lambda. It is the PM system that will create the binding, with a value, equal to the `number` variable, but only if `number` is `5`. The end result is the same and what is written will not be misleading - `n` is (equal to) `5`.

Moving to lambda-like identifiers introduction has a major benefit - we have syntax ready for all other identifiers!

Mutable and immutable bindings:

```
int number = ...;

void mutate(int&){...}

inspect (number) {
  n=__ => mutate(n); // FAILS, n is immutable by default!
  &n=__ => mutate(n); // SUCCEEDS, n is requested to be mutable
  ...
};
```

Binding to preexisting variables:

```
int number = ...;

int n;

inspect (number) {
  &n __ => mutate(n); // binding to existing variable with the same value
  as ::number
  // or just &n omitting the __
  ...
}
```

Compared to lambda side by side:

lambda capture

```
int x,y,z;  
  
[ x= ..., // new x  
  &y= ..., // new y, but reference  
  &z      // existing z reference  
{...};
```

PM bindings

```
int x,y,z;  
  
inspect (...) {  
    x= ... => // new x  
    &y= ... => // new y, but reference  
    &z ... => // existing z reference  
};
```

Arguably, assignment to existing variable in PM could look confusing when combined with other patterns. For example `&z 5` will be the syntax to assign to a preexisting `z` if its value is `5`. We have assignment without `=`, which is far from great. However, this use-case is rare, it is not even proposed in currently active proposals. As such, this paper is willing to accept this downside for the other benefits this approach brings.

As suggested, lambda captures and PM bindings are quite similar. In both minimal syntax is paramount. In both there are no uninitialized state. Both introduce the variables in a special, limited scope. The variables themselves in both cases have the same restricted, concise requirements - to introduce a value name, to introduce a reference name, to reuse a value name. This is a bit different to regular variables, which have greater available options to choose from, like opting for a pointer. Here, in lambda and PM, we have only 3 options, covering our use cases in a more manageable, simple manner.

In fact PM bindings, because they are not manually assigned, will be more limited even than lambda captures as type conversion is not possible.

Of course the details b/w bindings and captures will not be the same, but this is mainly due to different lifetime and implementation requirements, not so much because semantic differences. In fact, there are no semantic difference b/w regular variables and captures and bindings, and this is why it is of benefit to not introduce a completely new syntax just to cover one of those. More over, it should be reminded, the capture syntax is not a new syntax, just a cut down version of the existing one. Adopting it for PM *will* see differences in the internal representation and it might no longer match the representation of a regular variable, or capture, written with the same syntax: Bindings might be represented by const references, not copies; mutable bindings might be represented by pointers, even views, not straight-as-written C++ references. None of those however are changes, that introduce a different meaning of the code, creating new abstractions which require a different form of expression.

Moving further, another significant gain by this approach is the fact, **we can name types directly!**

Because all identifiers so far were “marked” by additional tokens, we could finally have “unmarked” names for types:

```
inspect (...) {
  int __ => // match int
  ...
};
```

Having natural type selection is a fundamental benefit, resurrecting the “dream syntax” from a decade ago, one w/o special tokens like `<>` or additional language features like `as`.

Putting it all together, we have **a complete set with zero ambiguity**:

```
struct size{...};
size size;
inspect (...) {
  size => // match the ::size type
  size=> // new `size` binding, hiding ::size
  &size=> // new `size` binding, hiding ::size, mutable
  &size => // assign to the ::size variable
  ==size => // match sizes, same as ::size
  <=size => // match sizes, less-than-or-equal to ::size
  >=size => // match sizes, greater-than-or-equal to ::size
  > size => // match sizes, greater then ::size
  < size => // match sizes, less then ::size
  ...
};
```

§ 4 Proposal

The above study is not proposed in its entirety or detail. In particular, *binding to existing variables, as well as the greater-then and less-then comparisons are **not** proposed*. This proposal considers these to be future direction or at least future proofing for not writing ourselves into a corner.

With that out of the way, **proposed identifiers are**:

- `id` - type name
- `id =` - immutable binding
- `&id =` - mutable binding
- `== id` - variable for equality comparison

Proposed is also a slightly different, more user-friendly version of the above specification. In particular, having to compare to a value using `==` can get verbose and might seem atypical. Similarly, but to a lesser degree, having single identifier to mean a type name might be confusing:

```
struct size{...};
size size;

inspect (...) {
```

```

size      => //< There is a certain expectation, this is a value,
           not a type, we compare against!
[size, size] => //< Here as well
};

```

To account for that, **proposed** is to have the `==` be *optional* if it is the last pattern, which is often the case.

“Last pattern” is the one that is matched after all other patterns in the current branch of the PM tree structure are matched.

For example the pattern `[x=5, x]` is a tree structure as follows:

1: match layout of two items.

2a: bind value of the first item to an `x`. **2b:** test second value for equality to `x`.

2a.1: test first value for equality to 5.

Both **2b** and **2a.1** are last patterns, because it is where each branch, **2a** and **2b**, ends.

Further, to avoid ambiguity, the type name must never be the last pattern. In practice this means, to match on type alone, one must follow up its name with another pattern, like

`__ :`

```

struct size{...};
size size;

inspect (...){
  size __      => //< Size is definitely a type
  size        => //< Value matched, "as expected"
  [size, size] => //< Values here as well*
  [==size, ==size] => //< Same as above, more verbose, but more clear?

  size size    => //< also valid, type is size, value equals the
                ::size variable
  size (==size) => //< same as above, but more clear
};

// *assuming some hypothetical structure, deconstructed into 2 sizes

```

There are cases however, where `==` is still required:

```

struct Size{...};
Size size;

inspect (...){
  // size[w=, h=] => // Error, no type named size
  // Size size[w=, h=] => // Same, also an error (attempt for two type
  // matches one after another)
  ==size[w=, h=] => // OK, match ::size
};

```

Omitting the `==` here is not allowed because the pattern is *not* the last one. We continue with a deconstruction and binding the width and height to `w` and `h` respectively. In that

case an attempt is made to match on a type, named `size`, not a value. To request match by value instead, `==` must be used.

Likewise the type match *must not* be the last pattern:

```
struct Size{...};
Size size;

struct Rectangle { int x; int y; Size size; };
Rectangle rect = ...;

inspect (rect) {
    //  [__, __, size __] => // Error, no type named size
    [__, __, ==size __] => // OK, explicit value match
        [__, __, size] => // OK, last pattern, match value

    //  [__, __, Size] => // Error, no such value (type match must not
        be last)
        [__, __, Size __] => // OK, not last pattern, match type
    [__, __, Size [w=, h=]] => // OK, not last pattern, match type
};
```

Advanced example

```
int x = 5;

inspect (...){
    x      => // match ::x
    ==x    => // same as above, but explicit
    x=     => // new binding, hiding ::x
    x=x    => // new binding, match value ::x
    x==(x) => // same as above, but explicit
    int x  => // match int, match value ::x
    px= (int x=x) => // bind px to a polymorphic type, then match the type
                    to int, bind to int value named `x`, match value to ::x
    int &x=x    => // match int, bind to int value reference named `x`,
                    match value of ::x
    ...
};
```

§ 4.1 Conclusion

Presented here was an investigation of identifiers usage inside PM, with the hope all limitations and use-cases are accounted for, at least in principal.

Along that, a system of identifiers is presented, which accounts for all use cases, does not require new language syntax and is based on existing practices. A small, most useful subset is proposed as a way to handle identifiers in a practical PM implementation.

Last but not least, suggested is that, by using the described system, it will be possible to have type matching in a natural way by just naming the type. This is the preferred way in most PM frameworks, both in C++ (all initial proposals) and other languages.

— `int` - type matching (just state the type!)

- `x=` - immutable binding (**safe default** with minimal syntax)
- `&x=` - mutable binding (**explicit** when modifying the inspected object)

Future extension

- `&x` - assign to an existing variable

§ 4.2 More Examples

§ 4.2.1 Point type matching and deconstruction

This Proposal	Main Proposal	Is As Proposal
<pre> struct Point {...}; Point p; int x = 10; int y = 20; inspect (...) { Point [x, y] => //< Point type, at pos (::x, ::y) Point [x=, y=] => //< Point type, bind x, y read only Point [&x=, &y=] => //< Point type, bind x, y for read and write Point [x=, y] => //< Point type, at ::y pos, bind x pos to `x` Point [x, y=] => //< Point type, at ::x pos, bind x pos to `y` Point [x=x, y=y] => //< Point type, at pos (::x, ::y), bind x, y }; </pre>	<pre> struct Point {...}; Point p; int x = 10; int y = 20; inspect (...) { <Point> [x, y] => // N/A, always mutable <Point> let [x, y] => <Point> [let x, y] => <Point> [x, let y] => <Point> [let x=x, let y=y] => }; </pre>	<pre> struct Point {...}; Point p; int x = 10; int y = 20; inspect (...) { [is x, is y] as Point => // N/A, always mutable [x, y] as Point => [x, is y] as Point => [is x, y] as Point => [x is x, y is y] as Point => }; </pre>

§ 4.2.2 Explicitly mutable, implicitly immutable bindings

```

struct Person{ std::string name; uint age; };

void yearCycle(Person& p) {
  ...
  inspect (p) {
    [name=, &age=] => std::invoke([&]{
      age++;           // increase age
      if(name = "Bob") // Error, name is immutable
        sendHappyBirthdayToBob();
    });
  };
};

```

```
};
}
```

§ 4.2.3 Pattern matching class hierarchies

```
class Animal { public: virtual void speak() const = 0; };

class Cat : public Animal {
public:
    virtual void speak() const override { std::cout << "Meow from " <<
        name << std::endl; }

    std::string name;
};

class Crow : public Animal {
public:
    virtual void speak() const override { std::cout << "Caaaw!" <<
        std::endl; }
};

const char* fluffy = "Fluffy";
```

This Proposal

Main Proposal

Is As Proposal

```
void listen(const
    Animal &a) {
    inspect (a) {
        Cat c=[fluffy] =>
            // use cat c
        Cat [name=] => //
            use name
        Crow c= => // use
            crow c
    };
}
```

```
void listen(const
    Animal &a) {
    inspect(a) {
        <Cat> (at!)[let c,
            [fluffy]] => //
            use cat c
        <Cat> [name] => //
            use name
        <Crow> let c => //
            use crow c
    };
}
```

```
void listen(const
    Animal &a) {
    inspect (a) {
        c as Cat is [_ is
            fluffy] => //
            use cat c
        c as Cat is [name is
            _] => // use
            name
        c as Crow => // use
            crow c
    };
}
```

§ 4.2.4 Extreme example

The following example extremity stems for the fact, both value matching and binding are assumed to be commutative. If this is not the case, then the patterns must appear in some predefined order. Composability is out of scope for this paper

```
struct Size{...};
Size size;

inspect (...){
    Size size [w=, h=] => //< type is ::Size, value equals ::size,
        w and h are introduced
    Size [w=, h=] size => //< same as above
```

```
Size sz= [w=, h=] size =>    //< same as above, but we also bind the
    size value to `sz`
Size size [w=, h=] sz= =>    //< same as above
Size sz= [w=, h=] ==size => //< same as above (more clear)

sz= [w=, h=] ==size =>      //< same as above, but valid for any
    type, not just ::Size
==size [w=, h=] sz= =>      //< same as above
};
```

1. Pattern Matching: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1371r3.pdf>↵
2. Pattern Matching Discussion for Kona 2022: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2688r0.pdf>↵
3. Pattern matching using is and as: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2392r1.pdf>↵
4. Pattern Matching for C++: <https://www.stroustrup.com/pattern-matching-November-2014.pdf>↵
5. Pattern Matching and Language Variants: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0095r1.html>↵